Resource Isolation in MoQT Relays

Progress Report II - Group 16

Kaustubh Bhal, Shradha Gandotra, Rudra Kanani, Sanat Dhanyamraju, Alan Zhu

November 10, 2026

Our project looks at how a Media over QUIC Transport (MoQT) relay can share bandwidth and memory fairly across subscribers. Since Report I, we have built a local testbed, tested how stalled clients affect other subscribers, and implemented a way for them to return to current media without disconnecting. This report covers those results and our preparation for a comprehensive experiment on PACE ICE.

1. Problem Definition

A MoQT relay receives media from a publisher and forwards it to many subscribers. Each subscriber has its own QUIC connection, but the connections share the relay's memory and outgoing bandwidth. When one subscriber stops keeping up, the relay may retain old media for it. Once that subscriber resumes, sending its backlog can compete with current media being delivered to other clients.

Report I proposed both fair bandwidth scheduling and limits on buffered media. We started with stalled subscribers because they give us a controlled way to study this interaction. Our immediate question was whether the relay could protect healthy clients while still allowing stalled clients to recover.

2. Motivation

Live media needs to arrive while it is still useful. A client receiving several seconds of old frames may have high throughput while remaining far behind the live stream. Sending that backlog also uses bandwidth that could carry current media to other subscribers.

Disconnecting every slow client would avoid some of this work, but would also interrupt clients experiencing temporary slowdowns. We therefore want to distinguish between ending a subscription and dropping media that is already too old to be useful. Our experiments compare those choices.

3. Challenges

The first challenge was making a client stall affect the relay. If we only slowed the final application reader, the client library could continue receiving and buffering data. We instead pause the client's MoQT session processing and use small receive buffers. As those buffers fill, the relay can no longer keep writing to that client. QUIC still processes packets and acknowledgments, so the connection remains open.

The second challenge was separating shared memory from memory used by individual connections. Several subscribers can reference the same media, so adding their queue sizes would count some payloads more than once. We traced the storage path to understand which data a stalled client can keep alive; the memory model is described below.

Finally, dropping old media must not make an incomplete Object look like a successful delivery. We changed the receiver to recognize intentional stream resets and continue reading later groups, while still treating malformed or truncated data as errors.

4. Methodology

Experimental platform and workload

We built the testbed around Cloudflare's moq-rs relay, pinned at MoQT draft-16 revision 3b2b584. It has three processes: a publisher, the relay, and a generator that opens independent subscriber connections. We began on one machine so we could debug the implementation and repeat experiments before moving to PACE. The local configuration is shown below.

The publisher generates synthetic media Objects, which are the units of data carried by MoQT. Each Object contains a sequence number, a generation timestamp, and a payload the receiver can verify. We send 30 Objects per second at about 4 Mbit/s. Every second starts a new group; in this workload, each group is sent on one QUIC stream, called a subgroup stream.

This continuous one-to-many workload lets us control the source rate and measure both complete delivery and how old the data is when it arrives. A speed test alone would not show whether a client is receiving useful live data or old backlog. Encoded-video replay, including Big Buck Bunny from Report I, remains a later experiment.

Setting	**Local platform and workload**
Hardware	Apple M2 Pro; 10 CPUs; 16 GiB RAM
Operating system	macOS 15.3.1, ARM64
Processes	One relay, one publisher, one subscriber generator; four runtime worker threads each
Source	30 Objects/s, 16,666 bytes each; approximately 4 Mbit/s; one new group per second
Subscribers	Independent QUIC sessions; 16 in policy comparisons, 16 or 64 in the initial baseline
Receive limits	64 KiB per connection, 32 KiB per stream, one incoming unidirectional stream
Bandwidth limit	80 Mbit/s of shared outgoing UDP payload, with a 64 KiB burst allowance
Replication	Five runs per condition, with condition order shuffled within each repetition

Relay modifications

We first ran the unchanged relay, then added a shared 80 Mbit/s outgoing UDP limit. Sixteen subscribers need about 64 Mbit/s of media before overhead, so normal delivery fits within the limit, while catch-up traffic can create contention. The limit includes QUIC headers, control

traffic, and retransmissions. It creates a controlled bandwidth bottleneck; it doesn’t decide which connection gets a fair share.

We implemented two ways to handle old work.

1) The ‘terminate’ policy ends a subscription if forwarding one of its subgroups takes more than three seconds.
2) The ‘skip’ policy uses the same threshold but resets the subscription's pending media streams and waits for a fresh group. The client permits this dropping and continues on its original connection and subscription.

The timer starts when the relay's downstream forwarder observes the subgroup and includes waiting to open a stream, sending data, and waiting for the final acknowledgment. It therefore limits how long forwarding work remains active, rather than directly measuring the client's distance from the live stream. Note: It’s not a per-connection memory cap.

We tested these changes with real QUIC connections, including expiry during stream opening, resets halfway through an Object, and delivery of a fresh group afterward. The Rust and client tests passed, as did 18 Python tests for the experiment and analysis tools. We also kept raw logs, source and binary hashes, and the commands needed to reproduce each accepted run.

How a stalled client affects relay memory

The relay uses memory in two main ways: storing media that subscribers share, and buffering data for each connection. In our testbed, the publisher generates media as it runs, so the relay does not load an original video file. It receives the Objects and stores them so several subscribers can read the same data.

When the relay passes data to QUIC, each subscriber's connection also needs buffers for bytes waiting to be sent or acknowledged. Figure 1 shows these regions separately because a limit on one does not necessarily limit the others. [6]

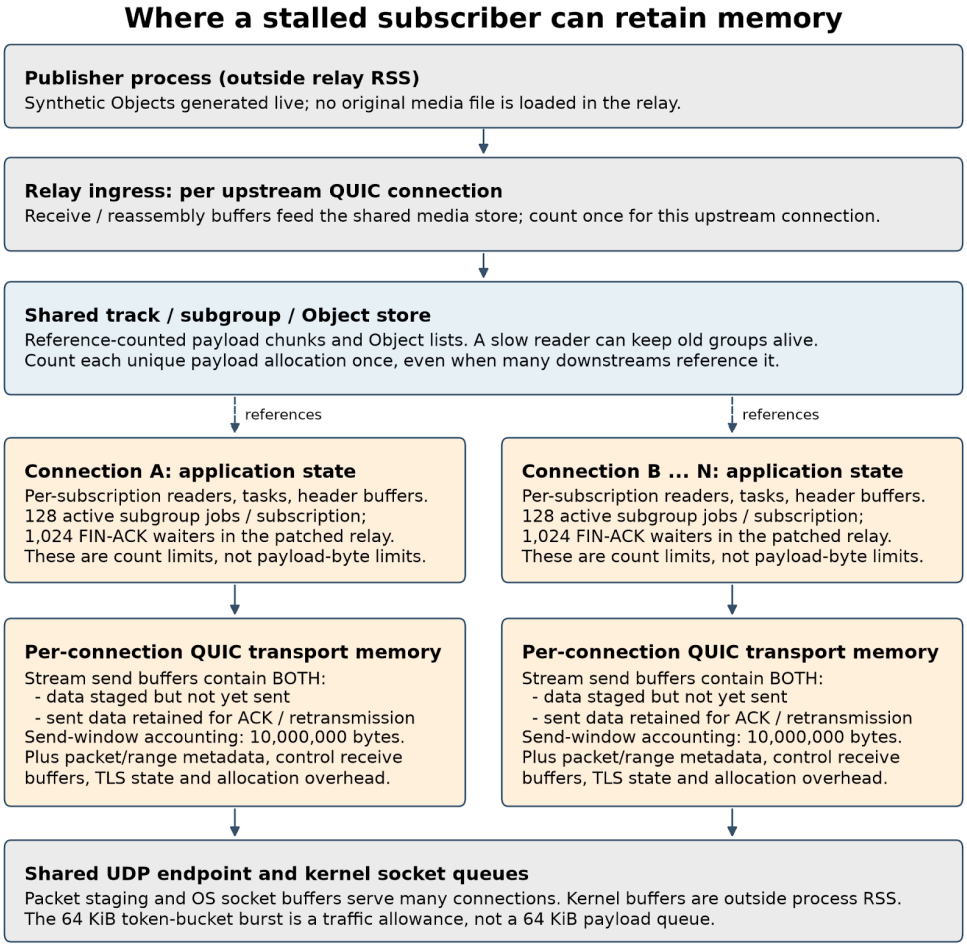


Figure 1. Where media is stored and which regions are shared. The numbers label configuration limits; they are not measurements of the maximum memory one connection consumes.

A stalled client can prevent shared media from being freed even after healthy clients have finished with it. However, that does not mean every stalled client creates another copy. At our 4 Mbit/s source rate, 15 seconds of media contains about 7.5 MB of payload. If eight clients are waiting for the same segment, the shared store holds that payload once. Clients waiting for different segments or different tracks can keep additional data in memory.

The amount one subscription can keep alive depends on the size of its groups. Our patched relay allows 128 active forwarding tasks per subscription. If each task holds one full group from our one-second-group workload, those groups contain about 64 MB of payload. This is an estimate for that specific situation, not an observed peak or a maximum for the whole connection. Larger groups would change the estimate. [6]

QUIC has a separate limit: the tested relay uses a 10 MB send window per connection. That limits its accounting of outstanding stream data, but it does not include the shared media described above or all of the connection's other memory. We therefore cannot say that a connection uses at most 10 MB. We have not yet established a total per-connection memory bound, and our three-second timeout does not provide one. [6, 7]

This is why the memory experiment needs a different scale check from the bandwidth experiment. We will first measure how much extra memory one stalled client keeps alive, then add clients that share a track and clients that receive different tracks. Those measurements will tell us how many connections are needed to create significant memory pressure. The current whole-process memory results appear in the evaluation section.

5. Related Work

PicNIC: Predictable Virtualized NIC studies how virtual machines sharing a server can interfere with one another's network performance. Its design combines fair queuing, receiver-driven congestion control, and sender-side shaping and accounting. The relevant lesson for our project is that controlling bandwidth alone does not isolate every shared resource. MoQT adds two complications: old media loses value, and several subscribers may share the same payload. Our current work examines those complications through stale-media handling and memory ownership; it does not implement PicNIC's scheduler. [1]

Media over QUIC: Initial Testing, Findings and Results provides an early experimental study of MoQ. The authors' companion testbed includes feedback and bandwidth measurements. Our work focuses on a different part of the delivery path: what happens when subscribers sharing one relay stall and then catch up. We use synthetic Objects to isolate that behavior before introducing video encoding and playback. [2]

Efficient Fair Queuing Using Deficit Round-Robin provides the scheduling baseline proposed in Report I. DRR gives each backlogged flow a credit measured in bytes and carries unused credit into later rounds. We plan to use subscriber sessions as the flows. However, receiving a fair share of bytes does not necessarily bring a client back to live media, so we will compare DRR alone with DRR plus stale-media skipping. DRR has not yet been implemented. [3]

6. Evaluation Results and Next Steps

Measurements

We completed 115 local runs across three experiments. Each condition has five repetitions, with the order shuffled within each repetition. Unless stated otherwise, we report the median of the five runs.

We use two measures to check whether clients receive useful live media. Latency is the time from Object generation to a complete receiver read; P99 is the latency at or below which 99% of received Objects arrive. Timely goodput counts only payload delivered within 150 ms, our chosen live-delivery target. Clients that receive nothing or terminate remain in the throughput calculation.

For return to live experiments, one fresh Object is not enough to count as recovery. We require a full one-second window after resume in which every received Object meets the 150 ms target, at least 80% of expected Objects arrive, and no delivery gap exceeds three frame intervals. Recovery time marks the start of that window, so we need the following second of data to confirm it. We also report how many clients recover within the ten-second observation period.

From baseline behavior to recovery without disconnecting

Our first 20 runs tested the unchanged relay with 16 or 64 clients and no outgoing bandwidth cap. Healthy clients stayed close to the source rate, and we did not find any consistent degradation. Previously paused clients received about 10 Mbit/s while catching up with a 4 Mbit/s source, showing that much of their traffic was backlogged. This gave us a reason to test recovery when bandwidth was limited.

The next 20 runs used the 80 Mbit/s cap with 16 clients, in these cases either none or eight clients paused for 15 seconds. With a three-second termination policy, healthy clients' P99 latency during recovery fell from 190 to 11 ms. Peak relay RSS, the amount of physical memory used by the relay process, fell from 36.92 to 23.70 MiB. However, the 40 paused subscriptions across all five mixed runs were completely terminated. Ending them protected healthy clients but did not let the stalled clients recover.

We then tested the ‘skip’ policy to see whether we could keep healthy clients close to live while allowing stalled clients to recover. We compared three policies under the same 80 Mbit/s cap: no timeout, ‘terminate,’ and ‘skip.’ Here, the baseline means the capped relay with its timeout disabled.

In each mixed run, eight of 16 clients paused after five seconds, then had ten seconds to recover after resuming. We tested pauses of 0.25, 1, 2, and 4 seconds, with five repetitions for each policy and pause length. We also ran five healthy-only controls per policy, giving 75 runs in total. All conditions used the same relay and client builds. Table 1 compares recovery with the baseline and skip policies.

Pause Length	Baseline #recovered	Baseline recovery time	Skip Policy #recovered	Skip Policy recovery time
0.25 s	40/40	0.088 s	40/40	0.080 s
1 s	40/40	1.733 s	40/40	1.693 s
2 s	40/40	4.104 s	40/40	3.979 s
4 s	30/40	8.443 s	40/40	0.024 s

Table 1. Recovery across five runs, with eight paused clients per run. For each run, we take the median recovery time among clients that recovered, then report the median of those five values. The four-second baseline time excludes ten clients that did not recover within ten seconds.

The clearest benefit appeared with four-second pauses. The skip policy recovered all 40 stalled clients on their original connections and subscriptions, compared with 30 of 40 for the baseline. During the final five seconds of recovery, previously paused clients received about 4.00 Mbit/s of timely data with the skip policy, compared to 1.30 Mbit/s for the baseline. This means they were receiving current media at roughly the source rate again.

Healthy clients benefited too. During recovery, their P99 latency fell from 174 to 48 ms, and their timely goodput rose from 3.91 to 4.00 Mbit/s per client. The terminate policy brought P99 down further, to 13 ms, but ended all 40 paused subscriptions. The skip policy therefore gave us a way to reduce interference while keeping those clients connected. No healthy client was terminated.

The 24 ms median in Table 1 does not mean every client recovered that quickly. Across skip's five runs, the median recovery time ranged from 5 ms to 2.104 seconds; the slowest individual client took 2.398 seconds. Figure 2 shows the variation across runs.

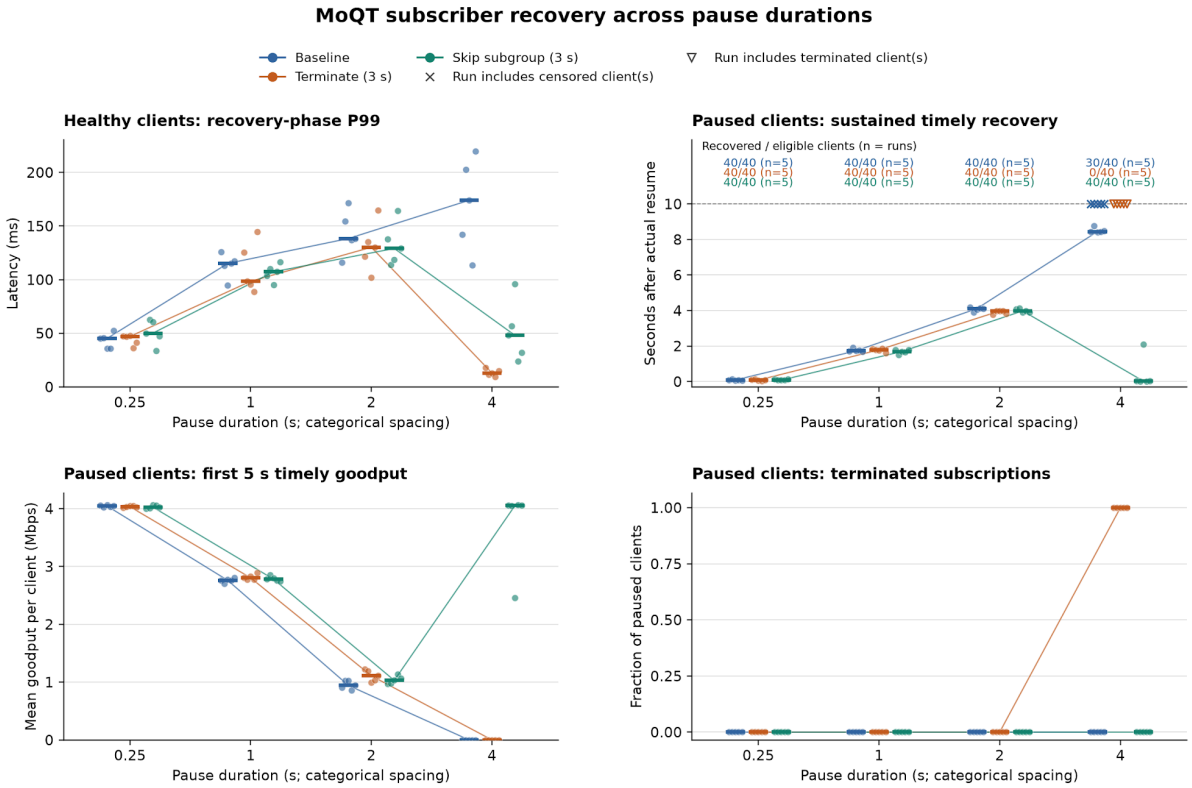


Figure 2. Individual runs and medians across pause lengths. Recovery counts include every paused client; recovery-time points include only clients that recovered.

What these results tell us

The shorter pauses help explain when the policy is useful. Neither timeout policy activated during pauses of 0.25, 1, or 2 seconds, so both behaved much like the baseline. All clients recovered, but a two-second pause still led to about four seconds of catch-up. During the four-second pauses, the forwarding timeout did activate, allowing skip to discard old work. Our fixed threshold can therefore help a longer stall while leaving a shorter one largely unchanged.

The memory result was smaller: for four-second pauses, median peak relay RSS during recovery fell from 26.98 MiB with the baseline to 23.81 MiB with skip. This measures the whole relay process, so it does not tell us how much memory each connection retained. It also does not demonstrate memory exhaustion. The results support skipping as a way to reduce interference from catch-up traffic; they do not yet show that we can enforce a memory limit for each client.

We checked that the recovery results came from valid runs. All 75 short-pause runs passed duration checks, covering 1,200 client observations, and the publisher stayed at 29.8–30.1 Objects/s during recovery. Connection and subscription identifiers confirmed that clients recovered without reconnecting. Earlier attempts with large timing gaps were excluded and kept separately.

These are still local results: the publisher, relay, and clients shared one machine. We also limited each client to one incoming media stream because overlapping groups exposed a recovery failure in the pinned library. Finally, our synthetic groups can be used independently, while a video player may need a keyframe before playback resumes. The current experiments demonstrate delivery of fresh Objects, not restored video playback.

Preparing the first PACE ICE experiment

PACE will let us separate the relay's resources from the machines generating its traffic. We have started work on a four-node launcher: one relay, one publisher, and two client generators that split the subscribers evenly. It requests eight CPUs and 32 GiB of memory per node for 30 minutes. The workers passed a local check of startup, client completion, logging, and cleanup using real QUIC connections. This checks the launcher logic, but not the PACE environment. [4, 5]

The first PACE run will be a 20-second check with 16 clients, no pauses, and the stock relay. We will check that every client completes, delivered payloads are valid, the publisher maintains its intended rate, and the generators have CPU capacity to spare. We will also record the allocated hardware and check clock differences before using timestamps from separate machines to measure latency.

Once that check passes, we will compare zero versus eight paused clients: five seconds of normal delivery, a four-second pause, and ten seconds of recovery, repeated five times per condition. The eight stalled clients will stay on one generator for this comparison. The launcher currently supports the stock-relay baseline. To test our modified policies on PACE, we still need to connect the patched Linux builds and policy settings to the launcher and combine both generators' logs for analysis. No PACE experiment has been run yet.

Remaining work and project scope

Report I proposed both fair bandwidth scheduling and memory limits. Our progress so far addresses a narrower question: we can reproduce interference from stalled clients and reduce it by dropping stale media while keeping those clients connected. The testbed, two timeout policies, and repeated local experiments are complete, but they do not yet establish fairness across sessions or memory isolation.

The largest remaining implementation tasks are DRR and memory budgets. DRR will let us compare fair bandwidth allocation alone with fair allocation plus skipping under the same cap. For memory, we first need to measure shared retained payloads and per-connection allocations separately, as described in Section 4. We can then set and test limits on bytes retained for each session and for the relay as a whole, and choose a client count that creates meaningful memory pressure.

We will prioritize those two questions and moving the policy comparisons to PACE. Encoded video, multiple tracks, and upstream subscription sharing would test how well the approach applies to other workloads; they are further evaluation, rather than prerequisites for the first PACE experiment.

References

1. Kumar et al. **PicNIC: Predictable Virtualized NIC.** SIGCOMM 2019. Paper.

2. Gurel et al. **Media over QUIC: Initial Testing, Findings and Results.** MMSys 2023. DOI; authors' testbed.

3. Shreedhar and Varghese. **Efficient Fair Queuing Using Deficit Round-Robin.** IEEE/ACM Transactions on Networking, 1996. Paper.

4. Georgia Tech PACE. ICE job guide and storage guide, linked from the GT teaching tutorial.

5. SchedMD. sbatch and srun documentation.

6. Project memory audit: ownership, source anchors, limits, and scale calculations for the tested relay.

7. Quinn-proto 0.11.13: transport configuration and send-buffer implementation.

Project repository